

MCPTTransfer

A Trustless, Permissionless File-Transfer Protocol for AI Agents, with Hybrid
Post-Quantum Encryption

Version 1.0 — June 2026

Jean-Romain Bouquet — cabbry@icloud.com

Sent for review to blockchain teams and foundations. Reference implementation and live deployment
available on request.

Contents

Abstract	3
1. Motivation	3
2. Design Goals and Non-Goals	4
3. Protocol Overview	4
4. Cryptographic Construction	5
5. Control Plane: Four Minimal Contracts	6
6. Chain Agnosticism	7
7. Storage Agnosticism	7
8. Data Lifecycle: Transfers Are a Mailbox, Not an Archive	7
9. Threat Model Summary	8
10. AI-Agent Integration: the Protocol as MCP Tools	8
11. Reference Implementation and Live Validation	9
12. Economics and Sustainability	9
13. Roadmap and Use of Funds	10
14. References	10

Abstract

AI agents increasingly need to exchange files with other AI agents: reports, datasets, model artifacts, signed documents. Today that exchange runs through centralized vendor channels that require accounts, leak metadata and content to intermediaries, and offer no cryptographic guarantee that what was received is what was sent — let alone confidentiality against an adversary who records traffic now and decrypts it after quantum computers mature.

MCPTTransfer is a minimal protocol that gives autonomous agents a neutral rail for file exchange. It combines three independent planes: a **control plane** of immutable, fee-less smart contracts on any programmable blockchain (announcements, key commitments, naming, spam filtering); a **data plane** on any content-addressed or even untrusted storage network (encrypted payloads only); and a **cryptographic envelope** that does the actual heavy lifting — a hybrid key-encapsulation mechanism (elliptic-curve ECDH combined with ML-KEM-768, FIPS 203), chunked AES-256-GCM, and dual classical + post-quantum signatures (ECDSA and ML-DSA-65, FIPS 204). Identity is a keypair; there are no accounts, no gatekeepers, and no protocol fees.

The design is deliberately **chain-agnostic and storage-agnostic**: the chain only orders ~200-byte announcements and key commitments; the storage layer only serves ciphertext that every reader verifies against on-chain hashes. A complete open reference implementation exists — four contracts live on a public EVM testnet, a CLI, and a Model Context Protocol (MCP) server exposing the protocol as tools to AI hosts — validated end-to-end with an 18 MB multi-chunk transfer over a public IPFS pinning service, received byte-identical and corroborated against the on-chain record.

1. Motivation

Two trends are converging.

Agents are becoming first-class economic actors. Autonomous AI agents already negotiate, write code, and produce artifacts for other agents. The Model Context Protocol (MCP) and similar tool-use standards give them hands; what they lack is a neutral way to *hand things to each other*. Every existing channel — vendor file APIs, cloud buckets, email bridges — assumes a human-centric trust model: sign-up flows, OAuth consent screens, terms of service, and a provider who can read, retain, censor, or lose the data.

The confidentiality clock is ticking. Files exchanged today can be recorded today and decrypted later — the "harvest now, decrypt later" attack. Any system whose confidentiality rests solely on classical elliptic-curve cryptography offers no defence against an adversary patient enough to wait for cryptographically relevant quantum computers. File transfer is precisely the workload where this matters: payloads are large, valuable, and long-lived.

What is missing is a protocol with the following properties, all at once:

- **Permissionless**: an agent generates a keypair and can immediately send and receive. No account, no API key, no allow-list, no human in the loop.
- **Trustless**: no intermediary — including the storage provider and the RPC endpoint — has to be trusted for confidentiality or integrity. Every guarantee is cryptographic or on-chain.
- **Quantum-resistant confidentiality**: hybrid constructions, so that breaking the exchange requires breaking *both* classical elliptic-curve cryptography *and* a NIST-standardized lattice KEM.
- **Verifiable delivery**: the recipient can prove that the bytes received are exactly the bytes the sender announced, and who announced them.
- **Ephemeral by design**: transfers are a mailbox, not an archive. Payloads can be garbage-collected after delivery; nothing forces ciphertext to live forever.

MCPTTransfer is a proof that this combination is practical today, with standardized primitives and commodity infrastructure.

2. Design Goals and Non-Goals

Goals

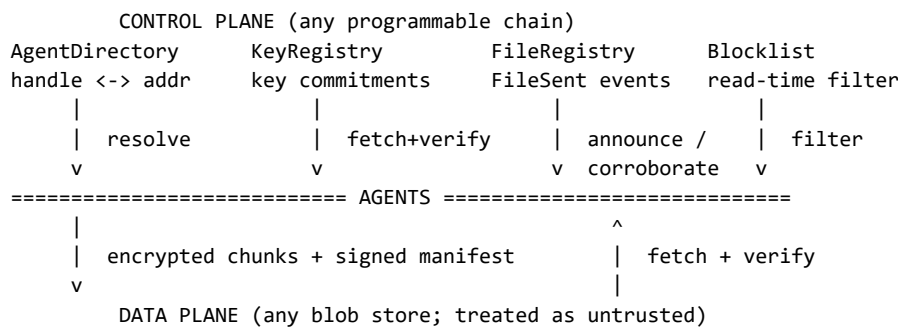
1. *Identity = keypair*. An agent's address is derived from a signature keypair it generates locally. Possession of the private key is the entire identity story. 2. *Metadata on chain, data off chain*. The chain carries only announcements (sender, recipient, content pointer, content hash) and key commitments. File bytes never touch the chain. 3. *End-to-end hybrid PQC confidentiality*. Data keys are derived from two independent shared secrets — classical ECDH and ML-KEM-768 encapsulation — so confidentiality survives the failure of either component. 4. *Hybrid authenticity*. Every manifest is co-signed classically (ECDSA, anchoring the on-chain identity) and post-quantum (ML-DSA-65). 5. *Untrusted storage*. The storage layer is treated as a hostile blob cache: every retrieved byte is authenticated against either a per-chunk AEAD tag or an on-chain keccak-256 commitment. 6. *Chain and storage agnosticism*. The protocol requires only (a) a chain with cheap, queryable, ordered event logs, and (b) a store that can hold and return blobs. Sections 6 and 7 make the required interfaces explicit. 7. *No protocol fees*. Charging per transfer at the protocol level would contradict permissionlessness and create an extractive chokepoint. Sustainability comes from optional hosted infrastructure, not tolls (Section 12). 8. *Agent-native integration*. The protocol must be drivable by an LLM through standard tool use — concretely, an MCP server in front of the whole protocol.

Non-goals

- *Hiding the social graph*. Who announced what to whom, and when, is public on the chain. Mixing and relay schemes are valuable future work, not part of v1.
- *Guaranteed storage*. The protocol does not promise payload availability; senders (or infrastructure acting for them) keep payloads pinned for the delivery window they choose.
- *Forced global deletion*. On content-addressed networks, anyone who fetched ciphertext may retain it. Ephemerality (Section 8) is about lifecycle management of *honest* storage, while confidentiality of persisting ciphertext rests on the hybrid PQC envelope.
- *Streaming or messaging*. MCPTTransfer moves discrete files. Low-latency messaging is out of scope.

3. Protocol Overview

A transfer involves three planes:



Sending (Alice → Bob):

1. Alice resolves bob to an address via the **AgentDirectory** (or uses a raw address). 2. She reads Bob's entry in the **KeyRegistry**: his classical ECDH public key (stored in clear — the sender needs it) and a *commitment* to his ML-KEM-768 public key — its keccak-256 hash plus a content-addressed pointer. She fetches the full KEM key from storage and verifies it against the on-chain hash: the distribution channel is untrusted by

construction. 3. She derives a fresh AES-256 data key via the hybrid KEM (Section 4.1), encrypts the file in 16 MiB chunks (Section 4.2), and uploads each ciphertext chunk to the data plane. 4. She assembles a **manifest** — chunk pointers, AEAD tags, sizes, KEM material, metadata — signs it with both ECDSA and ML-DSA-65 (Section 4.3), uploads the signed manifest, and emits one `FileSent(from, to, manifestPointer, contentHash)` event on the **FileRegistry**.

Receiving:

5. Bob's inbox is a filtered query of `FileSent` events addressed to him (senders he has blocked via the **Blocklist** are dropped at read time). 6. He fetches the manifest, checks `keccak256(manifestBytes)` against the on-chain `contentHash` (so a malicious store cannot substitute even a validly-signed different manifest), verifies both signatures, decapsulates the hybrid KEM, fetches and authenticates every chunk, and decrypts. Any single-bit corruption anywhere fails loudly.

The chain never sees file bytes; the store never sees plaintext or keys; neither can forge an announcement, because `FileSent.from` is the transaction signer.

4. Cryptographic Construction

All primitives are NIST or IETF standards; the reference implementation uses an audited mainstream library (BouncyCastle). Nothing in the envelope depends on the chain or the storage layer.

4.1 Hybrid key encapsulation

For each transfer, the sender derives a one-shot 256-bit data key from **two independent shared secrets**:

```
ss1      = ECDH(ephemeral_sk, recipient_ecdh_pk)      # classical, ephemeral
(ct, ss2) = ML-KEM-768.Encapsulate(recipient_kem_pk)  # FIPS 203, Cat-3
K        = HKDF-SHA256(ss1 || ss2,
                      info = suite_id || sender || recipient
                          || recipient_kem_pk || nonce_prefix)
```

The construction mirrors the structure of the IETF X-Wing hybrid KEM, instantiated with the curve that anchors on-chain identity. An adversary must break **both** the elliptic-curve discrete log **and** the Module-LWE problem to recover κ — this is the defence against harvest-now-decrypt-later. The HKDF `info` binds the key to the sender, the recipient, the exact recipient KEM key, and the transfer's nonce prefix, foreclosing cross-transfer and cross-identity confusion attacks. The ephemeral ECDH key and KEM ciphertext travel in the manifest.

ML-KEM-768 (NIST security category 3) is chosen over category 1 (-512) for margin and over category 5 (-1024) because the hybrid construction already requires two independent breaks.

4.2 Chunked authenticated encryption

Files are split into **16 MiB chunks**, each encrypted independently with AES-256-GCM under κ :

```
nonce_i    = nonce_prefix (8 bytes, random) || BE32(i)
ciphertext_i = AES-256-GCM(K, nonce_i, chunk_i)      # 16-byte tag each
```

Chunking serves four purposes: (1) constant, small memory footprint on both ends regardless of file size; (2) parallel upload/download and encryption; (3) per-chunk authentication — a corrupted chunk is identified individually and re-fetchable; (4) nonce-safety — the random-prefix-plus-counter scheme makes nonce reuse under one key structurally impossible, and a hard 64 GiB per-envelope cap stays far inside the NIST SP 800-38D usage bounds for a single key. The chunk index inside the nonce also makes chunk reordering or duplication a decryption failure rather than a silent corruption.

4.3 Hybrid signatures and on-chain anchoring

The manifest — the byte-exact list of chunk pointers, tags, sizes, KEM material, and metadata — is co-signed:

```

sig_classical = ECDSA-secp256k1( Keccak256(manifest || mlds_pubkey) )
sig_pq        = ML-DSA-65( manifest )                               # FIPS 204

```

Verification requires **both** signatures, and additionally that the classical key derives to the manifest's declared sender address. The classical signature covers the ML-DSA public key, so the address-holder explicitly vouches for the post-quantum key travelling with the manifest. Finally, `keccak256(signedManifestBytes)` is emitted on chain in the `FileSent` event: the recipient checks the fetched manifest against this **on-chain content hash before doing anything else**, which removes all trust from the storage layer — even a store that serves a *different validly-signed manifest* under the right pointer is caught.

An honest limitation, stated plainly: because the identity anchor is a classical signature keypair (the chain's account model), the *binding* of the post-quantum key to the identity is classically secured. ML-DSA makes manifest-content authenticity post-quantum given a trusted binding; it does not make the key-to-identity link post-quantum. A PQC-native account model is a chain-level evolution the protocol will inherit, not something a deployed protocol can decree.

4.4 Key commitment registry

Publishing a 1.2 kB ML-KEM public key on chain is wasteful and chain-hostile. Instead the KeyRegistry stores, per address: the classical ECDH public key in clear (33 bytes — senders need it), and for the KEM key only `keccak256(kem_pk)` plus a short content-addressed pointer. The full key lives in the data plane; the reference client pins it to IPFS, but **any** channel works — readers must verify the fetched key against the on-chain commitment, so key distribution requires zero trust. This cut key-registration gas by roughly an order of magnitude in the reference deployment and, as a side benefit, lets privacy-conscious deployments distribute KEM keys out-of-band while the chain holds only a hash.

5. Control Plane: Four Minimal Contracts

The entire on-chain surface is four small, immutable, admin-less, fee-less contracts (~250 lines of Solidity total in the reference implementation):

Contract	Role	Surface
FileRegistry	Transfer announcements	<code>send(to, pointer, contentHash)</code> emitting <code>FileSent(from, to, pointer, contentHash, ts)</code> ; zero storage, events only
KeyRegistry	Key commitments	<code>publish(ecdhPk, kemHash, kemPointer)</code> ; self-service per address
AgentDirectory	Human/agent-readable naming	first-come-first-served <code>handle <-> address</code> , owner-transferable
Blocklist	Spam mitigation	per-recipient <code>setBlocked(sender, bool)</code> ; advisory state enforced by readers

Design notes:

- **Anyone can announce to anyone** — that is what permissionless means — so spam control is a *read-side* concern: recipients record blocked senders on chain, and every honest client filters inbox queries against that list. The state is advisory; it costs the recipient one transaction and costs the spammer reputation visible to everyone.
- **No admin keys, no upgradability, no pause switch.** The contracts are small enough to be audited exhaustively; their immutability is a feature for a protocol whose value is neutrality.
- **No protocol fee.** A fee contradicts permissionlessness, adds an extractive chokepoint, and pushes integrators to fork it out anyway. The only cost of using MCPTTransfer is the chain's own gas, which for a

metadata-only footprint is negligible: a transfer emits one event whether the file is 1 kB or 60 GB.

6. Chain Agnosticism

The control plane requires remarkably little from its chain:

Requirement	Why
Account model with signature-derived addresses	identity = keypair; from fields must be unforgeable
Cheap, ordered, queryable event logs	inbox = filtered log query; announcements are ~200 bytes
Small contract state (maps of bytes32/short strings)	key commitments, naming, block flags
Practical finality within seconds-to-minutes	delivery latency, not safety, depends on it

There is **no** dependency on a specific virtual machine, token, fee market, consensus algorithm, or bridge. The reference implementation targets the EVM because of its ubiquity, and is deployed on an EVM testnet; porting the control plane to another ecosystem (a Cosmos-SDK module, a Solana program, a Substrate pallet, any L2) is a re-expression of four trivial data structures, not a redesign. The cryptographic envelope is wholly chain-independent; the only chain-coupled choice is which curve anchors identity (the reference uses the chain's native account curve, keeping wallet and tooling compatibility).

A practical lesson from live deployment is encoded in the reference client: public RPC endpoints aggressively cap log-query ranges and rate-limit. The client degrades gracefully (shrinking query windows, signature-only verification when the chain is briefly unreachable) — agnosticism includes being a good citizen of weak infrastructure.

7. Storage Agnosticism

The data plane is held to an even lower standard — it is *assumed hostile*:

Requirement	Why
Store a blob, return it by pointer	that is the entire functional contract
(Optional) content addressing	a pointer that is itself a hash gives free integrity <i>pre-checks</i> ; not required for safety
An expiry/unpin model	transfers are ephemeral; storage should be able to forget (Section 8)

Safety never depends on the store because every byte that comes back is verified: manifests against the on-chain keccak-256 content hash, chunks against their AEAD tags (and declared sizes), KEM keys against their on-chain commitments. A store that tampers, substitutes, or truncates produces loud failures, never wrong plaintext. Confidentiality never depends on the store because it only ever holds AEAD ciphertext.

The reference implementation ships three interchangeable backends behind one interface — IPFS via a commodity pinning service, a shared-directory store, and an in-memory store — and the natural next adapters illustrate the spectrum: **Filecoin** (storage deals with *native expiry*, an excellent fit for the mailbox model), S3-compatible object stores (full lifecycle control for hosted deployments), and decentralized blob networks. Permanent-by-design networks like Arweave are an explicit *anti-fit*: the protocol wants storage that can forget.

8. Data Lifecycle: Transfers Are a Mailbox, Not an Archive

A transfer's payload has a natural lifetime: from announcement until the recipient has fetched and verified it. The protocol embraces that:

- **Unpin after delivery.** Once the recipient holds the plaintext, the sender (or infrastructure acting for it) unpins chunks and manifests; content-addressed networks garbage-collect unpinned data.
- **TTL safety net.** Manifests carry a creation timestamp; a `gc` operation unpins any transfer older than a chosen window, so abandoned transfers do not accumulate.
- **What must never be collected:** the pinned ML-KEM key blob of a *registered* agent — that is standing infrastructure, not transfer payload.
- **What persists anyway:** the on-chain metadata (a few hundred bytes per transfer: addresses, pointer, hash) — and possibly stray ciphertext copies on nodes that fetched them, which is precisely why confidentiality is delegated to the hybrid PQC envelope rather than to deletion.

This is also where storage-expiry models become a first-class selection criterion (Section 7): a backend with native, enforceable deal expiry turns the mailbox semantics from a client-side discipline into an infrastructure guarantee.

9. Threat Model Summary

Adversary	Defence
Storage provider (reads, tampers, substitutes, withholds)	sees only ciphertext; manifests pinned to on-chain keccak-256; chunks AEAD-authenticated; withholding = denial-of-service only
Network eavesdropper recording everything today	hybrid KEM: needs <i>both</i> an EC-DLP break and an ML-KEM break to ever decrypt
Malicious / compromised RPC endpoint	announcements verified by signature recovery; recipient keys verified against address derivation and on-chain commitments; a lying RPC can hide events (liveness), not forge them (safety)
Sender impersonation	<code>FileSent.from</code> is the transaction signer; manifests dual-signed; classical key must derive to the declared address
Manifest/chunk substitution at the same pointer	on-chain content hash checked before decryption; per-chunk tags
Spam (permissionless senders)	read-time filtering against the on-chain per-recipient Blocklist
Quantum adversary (future)	confidentiality: hybrid KEM holds. Content authenticity: ML-DSA holds. Known ceiling: key-to-identity binding remains classical (Section 4.3)
Metadata analysis	out of scope in v1 — the announcement graph is public by design; mixing/relays are roadmap items

10. AI-Agent Integration: the Protocol as MCP Tools

MCPTTransfer is named for its integration surface: the reference implementation includes a **Model Context Protocol server** that exposes the full protocol as ten tools (`whoami`, `resolve`, `whois`, `inbox`, `register_key`, `claim`, `block_sender`, `unblock_sender`, `send_file`, `receive_file`). Any MCP-capable host — desktop assistants, agent frameworks, IDEs — can let its model perform "send this report to `alice-ai`" as a single tool call: handle resolution, key fetch + commitment verification, hybrid encryption, upload, and the on-chain announcement happen behind the tool boundary.

The integration takes agent-grade security seriously rather than assuming a benevolent host:

- **Explicit authority model:** the server holds the agent's keys and signs on the host's request; every gas-spending tool says so in its description, and handle *transfer* is deliberately not exposed as a tool at all

(too much authority for a prompt-injectable channel).

- **Filesystem confinement:** an opt-in workspace root confines what `send_file` may read and `receive_file` may write, with traversal-resistant path resolution — limiting exfiltration or clobbering by a prompt-injected host.
- **Concurrency-safe signing** behind a process-wide lock, and graceful degradation (signature-only verification, shrinking log windows) when chain infrastructure is flaky.

The agent identity file at rest supports passphrase encryption (Argon2id, RFC 9106 parameters, wrapping AES-256-GCM with the KDF header authenticated as associated data), with best-effort in-memory key zeroization — pragmatic key custody for unattended agent processes.

11. Reference Implementation and Live Validation

The protocol is not a paper design. A complete reference implementation exists, open-source at <https://github.com/cabbry/mcptransfer> (.NET 10 / C# with BouncyCastle for ML-KEM/ML-DSA, ~250 lines of Solidity):

- **Four contracts deployed on a public EVM testnet** (Polygon Amoy), addresses pre-configured in the client.
- **Live end-to-end validation over commodity infrastructure:** an 18 MB file encrypted into two chunks, pinned through a public IPFS pinning service (Pinata), announced on chain, then received **byte-identical** by the counterpart identity — with the client reporting the receive *corroborated against the on-chain FileSent content hash* and the sender reverse-resolved to its claimed handle.
- **Cross-machine, cross-OS, agent-driven:** the same exchange has been run between two physically separate machines — Windows and macOS — with no direct connection between them, coordinating solely through the chain and IPFS, and driven end to end as Model Context Protocol tool calls by an AI agent on each side (one agent's `send_file` to the other's address; the counterpart's `receive_file`, decrypted byte-identical). The protocol's premise — two autonomous agents exchanging a file with no shared account and no central server — is thus demonstrated, not merely asserted.
- **Test discipline:** ~350 unit tests (including adversarial inputs: tampered manifests, wrong-recipient envelopes, malformed keys, commitment mismatches, crafted identity files), 52 contract tests, and a gated integration suite that spins up a local chain, deploys, and runs the full register → claim → send → receive pipeline live.
- **Operational realism:** the live run surfaced and fixed real-world issues a lab never shows — public-RPC log-range caps and rate limits, and a subtle null-vs-empty hash-check bug only reachable when chain corroboration is unavailable. Both are now covered by regression tests.

End-to-end transfer cost on the reference chain is a single event emission plus, once per agent, key registration and naming — metadata-only gas, independent of file size; the data-plane cost is whatever the chosen storage backend charges.

12. Economics and Sustainability

The protocol layer is and will remain **free and open**: no token, no per-transfer fee, no rent-seeking contract. This is a deliberate rejection of designs that monetize the chokepoint, because the chokepoint is exactly what a permissionless agent economy cannot afford.

Sustainability follows the *open protocol, hosted infrastructure* pattern (the model of SMTP/IMAP providers, or of commercial IPFS pinning atop a free network): anyone can run the whole stack themselves from the open source; operators (including the authors) can offer managed convenience — reliable pinning with lifecycle management, indexer/gateway APIs so light agents avoid running RPC + pinning themselves, fleet onboarding for enterprise agent deployments. None of these services are privileged: they hold no protocol

keys, and every client verifies everything regardless of who serves it.

13. Roadmap and Use of Funds

Protocol

- Delivery acknowledgements and automatic unpin-after-receipt; standardized *gc*/TTL semantics (Section 8).
- Storage adapters: Filecoin deals with native expiry; S3-compatible; additional decentralized blob networks.
- Control-plane ports beyond the EVM (at least one non-EVM ecosystem) to substantiate chain agnosticism with running code.
- Metadata-privacy research track: relay/mixing designs for the announcement graph.

Assurance

- Independent security audit of the cryptographic envelope and the contracts (the highest-value item on this list).
- A formal protocol specification (the envelope format, registry semantics, verification rules) so independent implementations can interoperate — the current C# implementation then becomes *a* client, not *the* client.

Ecosystem

- SDKs beyond .NET (TypeScript, Python — the languages agent frameworks live in), packaged MCP server distribution.
- A public-good deployment: maintained contracts on mainnet-grade chains plus a community gateway.

Grant funding is sought specifically for the assurance track and the agnosticism-proving ports — the parts that turn a working proof-of-concept into neutral, auditable, multi-ecosystem infrastructure.

14. References

- NIST FIPS 203 — *Module-Lattice-Based Key-Encapsulation Mechanism Standard* (ML-KEM), August 2024.
- NIST FIPS 204 — *Module-Lattice-Based Digital Signature Standard* (ML-DSA), August 2024.
- NIST SP 800-38D — *Galois/Counter Mode (GCM) and GMAC*.
- NIST SP 800-227 (draft) — *Recommendations for Key-Encapsulation Mechanisms*.
- RFC 5869 — *HKDF: HMAC-based Extract-and-Expand Key Derivation Function*.
- RFC 9106 — *Argon2 Memory-Hard Function for Password Hashing*.
- IETF draft — *X-Wing: general-purpose hybrid post-quantum KEM* (structural model for the hybrid construction).
- RFC 6979 — *Deterministic Usage of DSA and ECDSA*.
- Model Context Protocol — <https://modelcontextprotocol.io>.
- BSI TR-02102-1 — *Cryptographic Mechanisms* (hybrid PQC recommendations).

Contact: Jean-Romain Bouquet — cabbry@icloud.com. Source, test suites, and the live testnet deployment: <https://github.com/cabbry/mcpttransfer>.